

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Проектирование и реализация баз данных

Лабораторная работа № 2-23

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлуниин Александр Александрович

г. Санкт-Петербург 2026 г

Цель этапа: разработать программный скрипт для автоматизированного сбора данных из выбранного источника NHL API, их преобразования в формат, соответствующий спроектированной NoSQL-схеме, и последующей загрузки в MongoDB.

Используемые технологии:

- язык программирования: Python 3;
- библиотеки: requests для работы с API, pymongo для взаимодействия с MongoDB;
- СУБД: MongoDB Community Server;
- инструмент визуальной проверки: MongoDB Compass.

Процесс извлечения и подготовки данных был разделён на несколько этапов.

Получение данных о командах.

Сначала выполнялся запрос к franchise endpoint NHL API для получения списка действующих команд. На основе ответа формировались документы коллекции teams, содержащие идентификатор команды, название и год основания. Для полей city, arena, conference и division в скрипте использовался встроенный справочник действующих клубов NHL, что позволило сразу загружать коллекцию teams в полном виде без дополнительной постобработки.

Получение списка матчей.

Далее скрипт обращался к endpoint расписания матчей NHL API. В качестве входных дат использовались несколько фиксированных дат регулярного сезона 2023–2024: 2023-10-10, 2023-10-17, 2023-10-24, 2023-10-31. Такой подход был выбран намеренно для обеспечения воспроизводимости результата и стабильного объёма данных.

Получение детальной информации по матчам.

Для каждого game_id выполнялся отдельный запрос к endpoint gamecenter/{id}/landing, из которого извлекались дата матча, статус, идентификаторы домашней и гостевой команд, итоговый счёт, количество бросков, а также информация о заброшенных шайбах.

Трансформация данных.

Полученные JSON-ответы API преобразовывались в формат,

соответствующий схеме, разработанной на этапе 1.

Для коллекции games формировались поля:

- _id
- date
- status
- home_team_id
- away_team_id
- score
- team_stats
- goals

Нормализация статуса матча.

Так как API возвращал технические коды состояния матча (OFF, LIVE, PRE, FUT), в скрипте была реализована функция нормализации, преобразующая их в более понятные значения: Final, Live, Preview.

Формирование массива голов.

Из блока summary.scoring извлекались все голы матча. Для каждого гола сохранялись номер периода, время, идентификатор команды, имя автора гола и массив ассистов.

Загрузка данных в MongoDB.

После завершения этапа извлечения и преобразования данных сформированные документы были загружены в MongoDB в базу данных nhl_db.

Загрузка выполнялась в две коллекции:

- teams - справочная информация о командах NHL;
- games - основная коллекция с информацией о матчах.

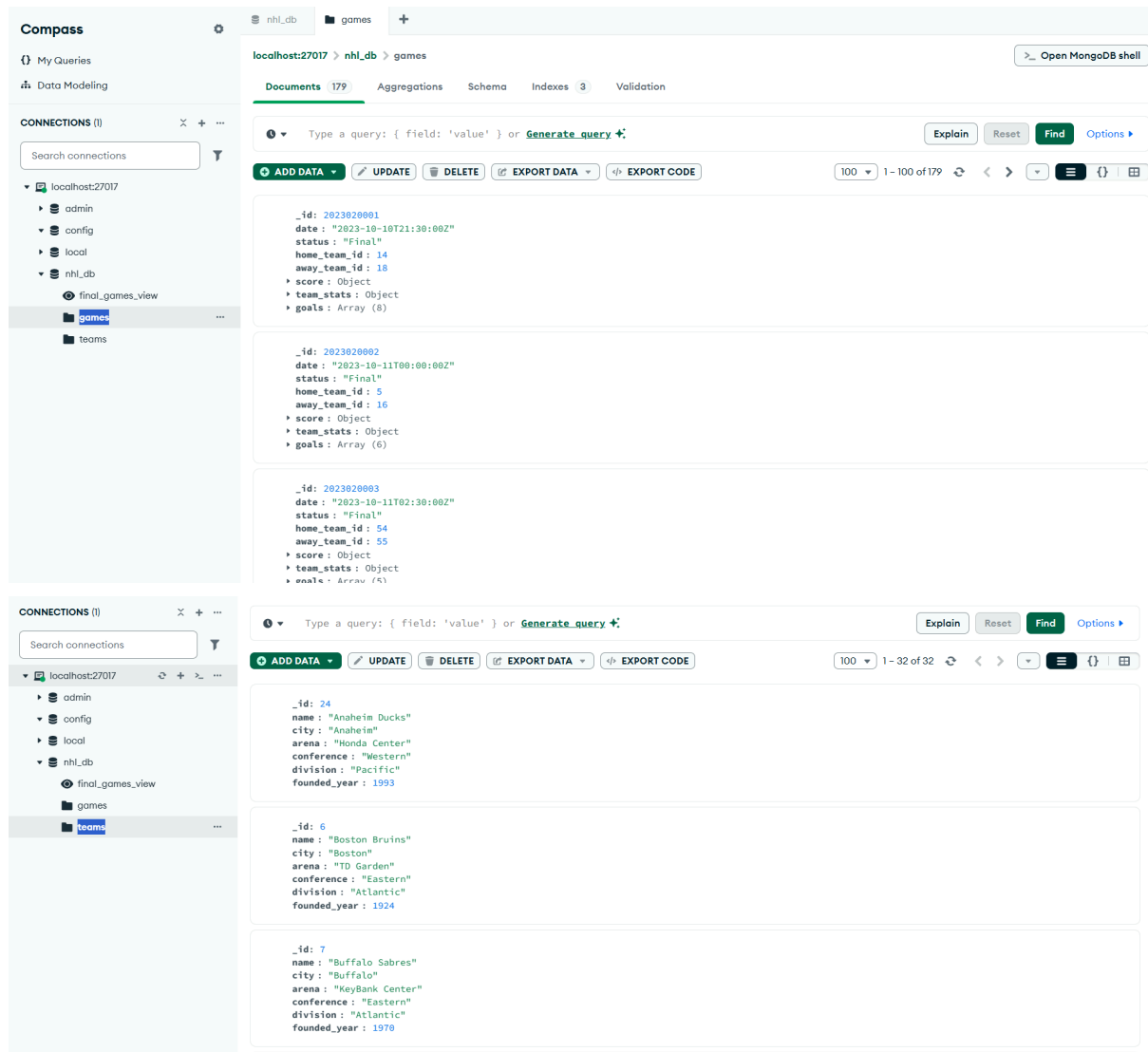
Перед загрузкой старые данные из коллекций удалялись, что позволяло повторно запускать скрипт без появления дубликатов. Для вставки использовался метод insert_many(), обеспечивающий пакетную загрузку документов.

В результате выполнения скрипта в базу было загружено:

- 32 документа в коллекцию teams;

- 179 документов в коллекцию games.

После вставки данные были проверены с помощью MongoDB Compass.



Создание представления (View)

В рамках дополнительного задания было создано представление `final_games_view`, основанное на коллекции `games`. Данное представление содержит только завершённые матчи, то есть документы, у которых поле `status` равно "Final".

Представление позволяет быстро получать список завершённых матчей без необходимости каждый раз задавать фильтр вручную.

Код создания представления:

```
db.command({  
  
  "create": "final_games_view",  
  
  "viewOn": "games",  
  
  "pipeline": [  
  
    {"$match": {"status": "Final"}}  
  
  ]  
  
})
```

The screenshot shows the MongoDB Data Modeling interface. On the left, the 'CONNECTIONS' panel lists a connection to 'localhost:27017' with a database named 'nhl_db'. Under 'nhl_db', there is a collection 'games' and a view 'final_games_view' which is currently selected. The main panel shows the 'Documents' tab with a search bar and a 'Generate query' button. Below the search bar, there are buttons for 'EXPORT DATA' and 'EXPORT CODE'. The results section displays three documents from the 'final_games_view' view, each representing a final game. Each document includes fields for '_id', 'date', 'status', 'home_team_id', 'away_team_id', 'score', 'team_stats', and 'goals'.

_id	date	status	home_team_id	away_team_id	score	team_stats	goals
2023020001	"2023-10-10T21:30:00Z"	"Final"	14	18	Object	Object	Array (8)
2023020002	"2023-10-11T00:00:00Z"	"Final"	5	16	Object	Object	Array (6)
2023020003	"2023-10-11T02:30:00Z"	"Final"	54	55	Object	Object	Array (5)

Создание индексов

Для ускорения выполнения запросов были созданы индексы в коллекции games.

Были добавлены следующие индексы:

- индекс по полю home_team_id;
- индекс по полю date.

Индекс по home_team_id предназначен для ускорения поиска матчей конкретной команды, а индекс по date — для ускорения выборок по дате проведения матчей.

Name & Definition	Type	Size	Usage	Properties	Status
▼ _id ↑	REGULAR	24.6 kB	3 (since Sat Apr 18 2026)	UNIQUE	READY
▼ home_team_id_1 ↑	REGULAR	20.5 kB	18 (since Sat Apr 18 2026)		READY
▼ date_1 ↑	REGULAR	20.5 kB	0 (since Sat Apr 18 2026)		READY

КОД ЭТАПА 2 ПАРСИНГ

```
import requests
from pymongo import MongoClient

MONGO_URI = "mongodb://localhost:27017/"
DB_NAME = "nhl_db"

TEAM_INFO = {
    24: {"city": "Anaheim", "arena": "Honda Center", "conference":
"Western", "division": "Pacific"},
    6: {"city": "Boston", "arena": "TD Garden", "conference":
"Eastern", "division": "Atlantic"},
```

```
7: {"city": "Buffalo", "arena": "KeyBank Center", "conference":  
"Eastern", "division": "Atlantic"},  
20: {"city": "Calgary", "arena": "Scotiabank Saddledome",  
"conference": "Western", "division": "Pacific"},  
12: {"city": "Raleigh", "arena": "Lenovo Center", "conference":  
"Eastern", "division": "Metropolitan"},  
16: {"city": "Chicago", "arena": "United Center", "conference":  
"Western", "division": "Central"},  
21: {"city": "Denver", "arena": "Ball Arena", "conference":  
"Western", "division": "Central"},  
29: {"city": "Columbus", "arena": "Nationwide Arena",  
"conference": "Eastern", "division": "Metropolitan"},  
25: {"city": "Dallas", "arena": "American Airlines Center",  
"conference": "Western", "division": "Central"},  
17: {"city": "Detroit", "arena": "Little Caesars Arena",  
"conference": "Eastern", "division": "Atlantic"},  
22: {"city": "Edmonton", "arena": "Rogers Place", "conference":  
"Western", "division": "Pacific"},  
13: {"city": "Sunrise", "arena": "Amerant Bank Arena",  
"conference": "Eastern", "division": "Atlantic"},  
26: {"city": "Los Angeles", "arena": "Crypto.com Arena",  
"conference": "Western", "division": "Pacific"},  
30: {"city": "Saint Paul", "arena": "Xcel Energy Center",  
"conference": "Western", "division": "Central"},  
8: {"city": "Montreal", "arena": "Bell Centre", "conference":  
"Eastern", "division": "Atlantic"},  
18: {"city": "Nashville", "arena": "Bridgestone Arena",  
"conference": "Western", "division": "Central"},  
1: {"city": "Newark", "arena": "Prudential Center",  
"conference": "Eastern", "division": "Metropolitan"},  
2: {"city": "Elmont", "arena": "UBS Arena", "conference":  
"Eastern", "division": "Metropolitan"},  
3: {"city": "New York", "arena": "Madison Square Garden",  
"conference": "Eastern", "division": "Metropolitan"},  
9: {"city": "Ottawa", "arena": "Canadian Tire Centre",  
"conference": "Eastern", "division": "Atlantic"},  
4: {"city": "Philadelphia", "arena": "Wells Fargo Center",  
"conference": "Eastern", "division": "Metropolitan"},  
5: {"city": "Pittsburgh", "arena": "PPG Paints Arena",  
"conference": "Eastern", "division": "Metropolitan"},  
28: {"city": "San Jose", "arena": "SAP Center at San Jose",  
"conference": "Western", "division": "Pacific"},  
55: {"city": "Seattle", "arena": "Climate Pledge Arena",  
"conference": "Western", "division": "Pacific"},  
19: {"city": "St. Louis", "arena": "Enterprise Center",  
"conference": "Western", "division": "Central"},  
14: {"city": "Tampa", "arena": "Amalie Arena", "conference":  
"Eastern", "division": "Atlantic"},
```

```

    10: {"city": "Toronto", "arena": "Scotiabank Arena",
"conference": "Eastern", "division": "Atlantic"},
    23: {"city": "Vancouver", "arena": "Rogers Arena",
"conference": "Western", "division": "Pacific"},
    54: {"city": "Las Vegas", "arena": "T-Mobile Arena",
"conference": "Western", "division": "Pacific"},
    15: {"city": "Washington", "arena": "Capital One Arena",
"conference": "Eastern", "division": "Metropolitan"},
    52: {"city": "Winnipeg", "arena": "Canada Life Centre",
"conference": "Western", "division": "Central"},
    53: {"city": "Salt Lake City", "arena": "Delta Center",
"conference": "Western", "division": "Central"},
}

START_DATES = ["2023-10-10", "2023-10-17", "2023-10-24",
"2023-10-31"]

client = MongoClient(MONGO_URI)
db = client[DB_NAME]
teams_col = db["teams"]
games_col = db["games"]

def normalize_status(status):
    return {
        "OFF": "Final",
        "LIVE": "Live",
        "PRE": "Preview",
        "FUT": "Preview"
    }.get(status, status)

def extract_first_year(season_id):
    if season_id is None:
        return None
    s = str(season_id)
    if len(s) >= 4 and s[:4].isdigit():
        return int(s[:4])
    return None

def get_teams():
    url = (
        "https://api.nhle.com/stats/rest/en/franchise"

        "?sort=fullName&include=firstSeason.id&include=lastSeason.id&inclu
de=teams"

```



```

)
data = requests.get(url, timeout=20).json()

teams = []
for item in data.get("data", []):
    last_season = item.get("lastSeason")
    if last_season is not None and last_season.get("id") is not
None:
        continue

    franchise_teams = item.get("teams", [])
    if not franchise_teams:
        continue

    team_id = franchise_teams[-1].get("id")
    if team_id is None:
        continue

    extra = TEAM_INFO.get(team_id, {})

    teams.append({
        "_id": team_id,
        "name": item.get("fullName"),
        "city": extra.get("city"),
        "arena": extra.get("arena"),
        "conference": extra.get("conference"),
        "division": extra.get("division"),
        "founded_year":
extract_first_year(item.get("firstSeason", {}).get("id"))
    })

    return teams

def get_game_ids():
    ids = []
    for start_date in START_DATES:
        url = f"https://api-web.nhle.com/v1/schedule/{start_date}"
        data = requests.get(url, timeout=20).json()

        for day in data.get("gameWeek", []):
            for game in day.get("games", []):
                ids.append(game["id"])

    return list(dict.fromkeys(ids))

def extract_goals(data):

```

```

home_team_id = data.get("homeTeam", {}).get("id")
away_team_id = data.get("awayTeam", {}).get("id")
home_abbrev = data.get("homeTeam", {}).get("abbrev")
away_abbrev = data.get("awayTeam", {}).get("abbrev")

goals = []

for period_block in data.get("summary", {}).get("scoring", []):
    period_number = period_block.get("periodDescriptor",
    {}).get("number")

    for goal in period_block.get("goals", []):
        team_abbrev = goal.get("teamAbbrev", {}).get("default")

        if team_abbrev == home_abbrev:
            team_id = home_team_id
        elif team_abbrev == away_abbrev:
            team_id = away_team_id
        else:
            team_id = None

        assists = []
        for assist in goal.get("assists", []):
            assist_name = assist.get("name", {}).get("default")
            if assist_name:
                assists.append(assist_name)

        goals.append({
            "period": period_number,
            "time": goal.get("timeInPeriod"),
            "team_id": team_id,
            "scorer_name": goal.get("name", {}).get("default"),
            "assists": assists
        })

    return goals

def get_games():
    games = []
    game_ids = get_game_ids()

    for i, game_id in enumerate(game_ids, start=1):
        try:
            url =
f"https://api-web.nhle.com/v1/gamecenter/{game_id}/landing"
            data = requests.get(url, timeout=20).json()

```

```

        games.append({
            "_id": data["id"],
            "date": data.get("startTimeUTC"),
            "status": normalize_status(data.get("gameState")),
            "home_team_id": data.get("homeTeam", {}).get("id"),
            "away_team_id": data.get("awayTeam", {}).get("id"),
            "score": {
                "home": data.get("homeTeam", {}).get("score",
0),
                "away": data.get("awayTeam", {}).get("score",
0)
            },
            "team_stats": {
                "home_shots": data.get("homeTeam",
{}).get("sog", 0),
                "away_shots": data.get("awayTeam",
{}).get("sog", 0),
                "home_pim_minutes": 0,
                "away_pim_minutes": 0
            },
            "goals": extract_goals(data)
        })

    print(f"[{i}/{len(game_ids)}] обработан матч {game_id}")

    except Exception as e:
        print(f"Ошибка для матча {game_id}: {e}")

    return games

def create_view():
    if "final_games_view" in db.list_collection_names():
        db.drop_collection("final_games_view")

    db.command({
        "create": "final_games_view",
        "viewOn": "games",
        "pipeline": [{"$match": {"status": "Final"}}]
    })

def get_stage(plan):
    if not isinstance(plan, dict):
        return None

    if plan.get("stage") == "IXSCAN":

```

```

        return "IXSCAN"

    for key in ("inputStage", "queryPlan", "winningPlan"):
        if key in plan:
            nested = get_stage(plan[key])
            if nested:
                return nested

    if "inputStages" in plan:
        for item in plan["inputStages"]:
            nested = get_stage(item)
            if nested:
                return nested

    return plan.get("stage")

def analyze_indexes():
    sample_game = games_col.find_one()
    if not sample_game:
        print("Нет данных для анализа индексов.")
        return

    test_home_team_id = sample_game["home_team_id"]

    try:
        games_col.drop_index("home_team_id_1")
    except Exception:
        pass

    try:
        games_col.drop_index("date_1")
    except Exception:
        pass

    explain_no_idx = db.command(
        "explain",
        {"find": "games", "filter": {"home_team_id":
test_home_team_id}},
        verbosity="executionStats"
    )

    print("\nАНАЛИЗ ИНДЕКСОВ И EXPLAIN:")
    print(f"Поиск БЕЗ индекса (home_team_id={test_home_team_id}):")
    print(f"    - Стратегия:
{get_stage(explain_no_idx['queryPlanner']['winningPlan'])}")
    print(f"    - Просмотрено документов:
{explain_no_idx['executionStats']['totalDocsExamined']}")

```

```

games_col.create_index("home_team_id")
games_col.create_index("date")

explain_idx = db.command(
    "explain",
    {"find": "games", "filter": {"home_team_id":
test_home_team_id}},
    verbosity="executionStats"
)

print(f"\nПоиск с индексом
(home_team_id={test_home_team_id}):")
print(f" - Стратегия:
{get_stage(explain_idx['queryPlanner']['winningPlan'])}")
print(f" - Просмотрено документов:
{explain_idx['executionStats']['totalDocsExamined']}")

if get_stage(explain_idx["queryPlanner"]["winningPlan"]) ==
"IXSCAN":
    print("\nИндекс используется: найден IXSCAN.")

def main():
    teams_col.delete_many({})
    games_col.delete_many({})

    if "final_games_view" in db.list_collection_names():
        db.drop_collection("final_games_view")

    try:
        games_col.drop_indexes()
    except Exception:
        pass

    print("Загрузка команд...")
    teams = get_teams()
    if teams:
        teams_col.insert_many(teams)
    print(f"Загружено команд: {len(teams)}")
    if teams:
        print("Пример команды:")
        print(teams[0])

    print("\nЗагрузка матчей...")
    games = get_games()
    if games:
        games_col.insert_many(games)

```

```

print(f"Загружено матчей: {len(games)}")
if games:
    print("Пример матча:")
    print(games[0])

print("\nПРОВЕРКА ДАННЫХ:")
print("Количество команд:", teams_col.count_documents({}))
print("Количество матчей:", games_col.count_documents({}))
print("\nОдна команда из БД:")
print(teams_col.find_one())
print("\nОдин матч из БД:")
print(games_col.find_one())

print("\nСОЗДАНИЕ ПРЕДСТАВЛЕНИЯ (VIEW):")
create_view()
print("Представление 'final_games_view' успешно создано.")

analyze_indexes()

print("\nГОТОВО.")
print("Проверь в MongoDB Compass:")
print(" - nhl_db -> teams")
print(" - nhl_db -> games")
print(" - nhl_db -> final_games_view")
print(" - nhl_db -> games -> Indexes")

if __name__ == "__main__":
    main()

```

Этап 3

1. Цель этапа

Целью третьего этапа является демонстрация навыков работы с запросами MongoDB к созданной базе данных nhl_db. В рамках этапа были реализованы выборки по одному и нескольким критериям, проекция полей, поиск по вложенному массиву документов и агрегационный запрос для анализа данных о матчах NHL.

Реализованные запросы

Запрос 1. Простой поиск по одному критерию

Задача: найти матчи, в которых команда с идентификатором 14 выступала в роли домашней команды.

Метод: использован метод `find()` с фильтрацией по полю `home_team_id`.

Запрос 2. Поиск с проекцией

Задача: отобрать матчи домашней команды 14, но вывести только основные поля документа.

Метод: использован метод `find()` с фильтрацией по полю `home_team_id` и проекцией полей `_id`, `date`, `status`, `score`.

Запрос 3. Поиск по вложенному массиву

Задача: найти матчи, в которых команда с идентификатором 14 забивала голы.

Метод: использован поиск по вложенному массиву документов `goals` с помощью dot notation: `"goals.team_id"`.

Запрос 4. Поиск по нескольким критериям

Задача: найти завершённые матчи, в которых команда 14 играла дома.

Метод: использован метод `find()` с одновременной фильтрацией по полям `home_team_id` и `status`.

Запрос 5. Агрегационный запрос

Задача: подсчитать количество матчей, в которых каждая команда выступала домашней.

Метод: использован Aggregation Pipeline, включающий этап `$group` для группировки по `home_team_id` и этап `$sort` для сортировки результатов по убыванию количества матчей.

Код запросов

```
from pymongo import MongoClient
```

```
MONGO_URI = "mongodb://localhost:27017/"
```

```
DB_NAME = "nhl_db"
```

```
client = MongoClient(MONGO_URI)

db = client[DB_NAME]

games_col = db["games"]


def run_queries():

    print("Запрос 1. Матчи, где команда 14 играла дома:")

    q1 = games_col.find(

        {"home_team_id": 14},

        {"_id": 1, "date": 1, "status": 1, "home_team_id": 1, "away_team_id": 1}

    )

    for doc in q1.limit(5):

        print(doc)


    print("\nЗапрос 2. Матчи команды 14 с проекцией (_id, date, status, score):")

    q2 = games_col.find(

        {"home_team_id": 14},

        {"_id": 1, "date": 1, "status": 1, "score": 1}

    )

    for doc in q2.limit(5):

        print(doc)
```



```
print("\nЗапрос 3. Матчи, где команда 14 забивала  
голы:")
```

```
q3 = games_col.find(  
    {"goals.team_id": 14},  
    {"_id": 1, "date": 1, "goals": 1}  
)
```

```
for doc in q3.limit(5):  
    print(doc)
```

```
print("\nЗапрос 4. Завершённые домашние матчи  
команды 14:")
```

```
q4 = games_col.find(  
    {"home_team_id": 14, "status": "Final"},  
    {"_id": 1, "date": 1, "status": 1, "score": 1}  
)
```

```
for doc in q4.limit(5):  
    print(doc)
```

```
print("\nЗапрос 5. Сколько матчей каждая команда  
сыграла дома:")
```

```
pipeline = [  
    {"$group": {"_id": "$home_team_id",  
"games_count": {"$sum": 1}}},  
    {"$sort": {"games_count": -1}}
```

```
]

q5 = games_col.aggregate(pipeline)

for doc in q5:

    print(doc)

if __name__ == "__main__":

    run_queries()
```

Результаты выполнения

Запрос 1. Матчи, где команда 14 играла дома:

```
{'_id': 2023020001, 'date': '2023-10-10T21:30:00Z', 'status': 'Final',
'home_team_id': 14, 'away_team_id': 18}
```

```
{'_id': 2023020055, 'date': '2023-10-19T23:00:00Z', 'status': 'Final',
'home_team_id': 14, 'away_team_id': 23}
```

```
{'_id': 2023020070, 'date': '2023-10-21T23:00:00Z', 'status': 'Final',
'home_team_id': 14, 'away_team_id': 10}
```

```
{'_id': 2023020086, 'date': '2023-10-24T23:00:00Z', 'status': 'Final',
'home_team_id': 14, 'away_team_id': 12}
```

```
{'_id': 2023020106, 'date': '2023-10-26T23:00:00Z', 'status': 'Final',
'home_team_id': 14, 'away_team_id': 28}
```

Запрос 2. Матчи команды 14 с проекцией (_id, date, status, score):

```
{'_id': 2023020001, 'date': '2023-10-10T21:30:00Z', 'status': 'Final', 'score':
{'home': 5, 'away': 3}}
```

{'_id': 2023020055, 'date': '2023-10-19T23:00:00Z', 'status': 'Final', 'score': {'home': 4, 'away': 3}}

{'_id': 2023020070, 'date': '2023-10-21T23:00:00Z', 'status': 'Final', 'score': {'home': 3, 'away': 4}}

{'_id': 2023020086, 'date': '2023-10-24T23:00:00Z', 'status': 'Final', 'score': {'home': 3, 'away': 0}}

{'_id': 2023020106, 'date': '2023-10-26T23:00:00Z', 'status': 'Final', 'score': {'home': 6, 'away': 0}}

Запрос 3. Матчи, где команда 14 забивала голы:

{'_id': 2023020001, 'date': '2023-10-10T21:30:00Z', 'goals': [{ 'period': 1, 'time': '09:48', 'team_id': 14, 'scorer_name': 'N. Kucherov', 'assists': ['V. Hedman', 'B. Point']}, { 'period': 2, 'time': '16:15', 'team_id': 18, 'scorer_name': 'R. O'Reilly', 'assists': ['F. Forsberg', 'R. McDonagh']}, { 'period': 3, 'time': '00:11', 'team_id': 18, 'scorer_name': 'J. Parssinen', 'assists': ['F. Forsberg', 'R. O'Reilly']}, { 'period': 3, 'time': '02:25', 'team_id': 14, 'scorer_name': 'N. Paul', 'assists': ['B. Point', 'S. Stamkos']}, { 'period': 3, 'time': '03:07', 'team_id': 14, 'scorer_name': 'B. Hagel', 'assists': []}, { 'period': 3, 'time': '08:48', 'team_id': 18, 'scorer_name': 'T. Novak', 'assists': ['P. Tomasino', 'G. Nyquist']}, { 'period': 3, 'time': '10:52', 'team_id': 14, 'scorer_name': 'N. Paul', 'assists': ['M. Sergachev', 'B. Point']}, { 'period': 3, 'time': '19:58', 'team_id': 14, 'scorer_name': 'N. Kucherov', 'assists': ['N. Paul']}]}

{'_id': 2023020023, 'date': '2023-10-14T23:00:00Z', 'goals': [{ 'period': 1, 'time': '05:11', 'team_id': 17, 'scorer_name': 'D. Sprong', 'assists': ['A. DeBrincat']}, { 'period': 1, 'time': '09:46', 'team_id': 14, 'scorer_name': 'S. Stamkos', 'assists': ['V. Hedman', 'N. Kucherov']}, { 'period': 1, 'time': '12:52', 'team_id': 14, 'scorer_name': 'B. Hagel', 'assists': ['V. Hedman']}, { 'period': 1, 'time': '14:35', 'team_id': 17, 'scorer_name': 'A. DeBrincat', 'assists': ['S. Gostisbehere']}, { 'period': 2, 'time': '02:01', 'team_id': 17, 'scorer_name': 'L. Raymond', 'assists': []}, { 'period': 2, 'time': '10:53', 'team_id': 14, 'scorer_name': 'S. Stamkos', 'assists': ['B. Hagel', 'A. Cirelli']}, { 'period': 2, 'time': '15:23', 'team_id': 17, 'scorer_name': 'J. Compher', 'assists': ['M. Seider', 'A. Copp']}, { 'period': 3, 'time': '04:44', 'team_id': 17, 'scorer_name': 'A. DeBrincat', 'assists': ['D. Larkin']}, { 'period': 3, 'time': '09:39', 'team_id': 14, 'scorer_name': 'V. Hedman',

'assists': ['N. Kuchеров', 'S. Stamkos']], {'period': 3, 'time': '19:59', 'team_id': 17, 'scorer_name': 'M. Seider', 'assists': ['J. Compher']}]}

{'_id': 2023020033, 'date': '2023-10-15T23:00:00Z', 'goals': [{'period': 1, 'time': '07:02', 'team_id': 9, 'scorer_name': 'V. Tarasenko', 'assists': ['R. Greig', 'J. Chychrun']}, {'period': 2, 'time': '01:00', 'team_id': 14, 'scorer_name': 'A. Cirelli', 'assists': ['D. Raddysh', 'T. Jeannot']}, {'period': 2, 'time': '08:48', 'team_id': 9, 'scorer_name': 'B. Tkachuk', 'assists': ['C. Giroux']}, {'period': 2, 'time': '13:00', 'team_id': 14, 'scorer_name': 'T. Jeannot', 'assists': ['M. Eyssimont', 'V. Hedman']}, {'period': 2, 'time': '19:01', 'team_id': 9, 'scorer_name': 'M. Joseph', 'assists': ['V. Tarasenko']}, {'period': 3, 'time': '12:53', 'team_id': 9, 'scorer_name': 'B. Tkachuk', 'assists': ['J. Sanderson', 'A. Zub']}, {'period': 3, 'time': '17:17', 'team_id': 9, 'scorer_name': 'T. Stützle', 'assists': ['B. Tkachuk', 'J. Korpisalo']}]}

{'_id': 2023020040, 'date': '2023-10-17T23:30:00Z', 'goals': [{'period': 1, 'time': '08:57', 'team_id': 7, 'scorer_name': 'Z. Girgensons', 'assists': ['T. Jost', 'M. Samuelsson']}, {'period': 1, 'time': '15:28', 'team_id': 7, 'scorer_name': 'J. Skinner', 'assists': ['O. Power', 'A. Tuch']}, {'period': 2, 'time': '17:58', 'team_id': 14, 'scorer_name': 'B. Hagel', 'assists': ['A. Cirelli', 'T. Jeannot']}, {'period': 3, 'time': '19:53', 'team_id': 14, 'scorer_name': 'B. Hagel', 'assists': ['B. Point']}, {'period': 4, 'time': '01:46', 'team_id': 7, 'scorer_name': 'D. Cozens', 'assists': ['J. Greenway', 'R. Dahlin']}]}

{'_id': 2023020055, 'date': '2023-10-19T23:00:00Z', 'goals': [{'period': 1, 'time': '11:10', 'team_id': 14, 'scorer_name': 'N. Paul', 'assists': ['S. Stamkos']}, {'period': 2, 'time': '00:14', 'team_id': 23, 'scorer_name': 'B. Boeser', 'assists': ['J. Miller']}, {'period': 2, 'time': '01:02', 'team_id': 23, 'scorer_name': 'T. Myers', 'assists': ['E. Pettersson']}, {'period': 2, 'time': '15:43', 'team_id': 14, 'scorer_name': 'M. Eyssimont', 'assists': ['C. Sheary', 'N. Perbix']}, {'period': 2, 'time': '16:52', 'team_id': 14, 'scorer_name': 'N. Kuchеров', 'assists': ['B. Hagel', 'D. Raddysh']}, {'period': 3, 'time': '04:42', 'team_id': 14, 'scorer_name': 'N. Kuchеров', 'assists': ['N. Paul', 'B. Point']}, {'period': 3, 'time': '18:11', 'team_id': 23, 'scorer_name': 'J. Miller', 'assists': ['F. Hronek', 'E. Pettersson']}]}

Запрос 4. Завершённые домашние матчи команды 14:

{'_id': 2023020001, 'date': '2023-10-10T21:30:00Z', 'status': 'Final', 'score': {'home': 5, 'away': 3}}

{'_id': 2023020055, 'date': '2023-10-19T23:00:00Z', 'status': 'Final', 'score': {'home': 4, 'away': 3}}

{'_id': 2023020070, 'date': '2023-10-21T23:00:00Z', 'status': 'Final', 'score': {'home': 3, 'away': 4}}

{'_id': 2023020086, 'date': '2023-10-24T23:00:00Z', 'status': 'Final', 'score': {'home': 3, 'away': 0}}

{'_id': 2023020106, 'date': '2023-10-26T23:00:00Z', 'status': 'Final', 'score': {'home': 6, 'away': 0}}

Запрос 5. Сколько матчей каждая команда сыграла дома:

{'_id': 54, 'games_count': 8}

{'_id': 4, 'games_count': 7}

{'_id': 7, 'games_count': 7}

{'_id': 9, 'games_count': 7}

{'_id': 17, 'games_count': 7}

{'_id': 2, 'games_count': 7}

{'_id': 29, 'games_count': 7}

{'_id': 15, 'games_count': 7}

{'_id': 14, 'games_count': 6}

{'_id': 10, 'games_count': 6}

{'_id': 23, 'games_count': 6}

{'_id': 22, 'games_count': 6}

{'_id': 28, 'games_count': 6}

{'_id': 5, 'games_count': 6}

{'_id': 8, 'games_count': 6}

{'_id': 1, 'games_count': 6}
{'_id': 30, 'games_count': 6}
{'_id': 6, 'games_count': 6}
{'_id': 18, 'games_count': 5}
{'_id': 53, 'games_count': 5}
{'_id': 24, 'games_count': 5}
{'_id': 52, 'games_count': 5}
{'_id': 13, 'games_count': 5}
{'_id': 26, 'games_count': 5}
{'_id': 25, 'games_count': 5}
{'_id': 19, 'games_count': 5}
{'_id': 55, 'games_count': 5}
{'_id': 20, 'games_count': 4}
{'_id': 16, 'games_count': 4}
{'_id': 21, 'games_count': 3}
{'_id': 3, 'games_count': 3}
{'_id': 12, 'games_count': 3}